EM7 Process Manager

Tags	EM7 Process Manager, memory limits, segfaults, enabling core dumps, gdb, proc_mgr.py, drbd, HA
Team Owner	Sys Eng
Status	ACTIVE
Last Modified	Oct 13, 2020

- Introduction
- Diagnosis and Mitigation
  - is proc_mgr.py running
  - process memory errors and segfaults: raising process memory limits
  - segfaults: enabling core dumps
    - for manually run processes
    - for processes started via systemd
  - analyzing core dump files
  - Oracle does not publish the debuginfos package we need to debug python
  - a procedure to compile python debug packages
  - segfaults: faulthandler.enable()
  - running in debug mode and gathering logs
  - Process Manager Hangs after drbd HA switchover
- Theory
  - systemctl details
  - process memory limits
  - Time Factor
  - Stderr for scheduled processes (/data/logs/X.py.log files and how they grow without limit)
- Post Mortem Articles
  - 2025-06-24 EM-74711 EM7 service active but not functional after HA failover
  - 2024-08-20 EM-66942: em7 service gets hung after HA failover, doesn't restart for 15 minutes delaying other necessary services from starting

- 2022-11-17 EM-54877 Proc_mgr hung in DE / RDS system on multiple clients
- 2022-11-11 NIH segfault topoplogy crunch
- 2020-05-02 Symmetry: RBA proc fails II
- 2020-01-17 Symmetry: RBA proc fails
- 2020-02-19 eLoyalty: std dev alerting and DCM deadlocks

- Known Issues
  - EM-34767: blocking subshell call to systemctl start

Introduction

The proc_mgr.py process starts most other processes (any process that has a row in the table master.system_settings_processes), and, among other things, sets process limits (like memory usage). This guide is both about debugging proc_mgr and other processes started by it, when the problem with the other process is under the control of proc_mgr (if another process were to run out of memory, for example).

Diagnosis and Mitigation

is proc_mgr.py running

proc_mgr.py is controlled via systemd (as service "em7") so you can check its status with:

```
systemctl status em7
```

a normal response would look like this

```
● em7.service - EM7 Process Manager
   Loaded: loaded (/usr/lib/systemd/system/em7.service; enabled; vendor preset: disabled)
   Active: active (running) since Wed 2020-01-29 16:04:46 UTC; 1 weeks 2 days ago
 Main PID: 23747 (python2.7)
   CGroup: /system.slice/em7.service
           └─23747 /usr/bin/python2.7 /opt/em7/backend/proc_mgr.py

Feb 07 16:50:12 cdb sudo[24105]: s-em7-core : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/bin/systemctl status em7_automation
Feb 07 16:50:12 cdb sudo[24116]: s-em7-core : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/bin/systemctl status em7_ctaskman
Feb 07 16:50:12 cdb sudo[24127]: s-em7-core : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/bin/systemctl status em7_lfpulld
Feb 07 16:50:12 cdb sudo[24133]: s-em7-core : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/bin/systemctl status em7_mfpulld
Feb 07 16:50:13 cdb sudo[24137]: s-em7-core : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/bin/systemctl status em7_mailparse
Feb 07 16:50:13 cdb sudo[24151]: s-em7-core : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/bin/systemctl status em7_metrics
Feb 07 16:50:13 cdb sudo[24155]: s-em7-core : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/bin/systemctl status nginx
Feb 07 16:50:13 cdb sudo[24159]: s-em7-core : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/bin/systemctl status em7_reporter.target
Feb 07 16:50:13 cdb sudo[24165]: s-em7-core : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/bin/systemctl status em7_patch_manager
```

```
Feb 07 16:50:13 cdb sudo[24169]: s-em7-core : TTY=unknown ; PWD=/ ; USER=root ; COMMAND=/bin/systemctl status em7_scheduler
```

If it is not running, you can try restarting it with:

```
systemctl restart em7

```

although that would not normally work as the service script is written to keep proc_mgr.py running at all times so if its not running there is a problem preventing it from starting.

process memory errors and segfaults: raising process memory limits

When an EM7 process exits with a memory error (you can see this in silo.log as one of the debug messages proc_mgr.py writes) or with a segmentation violation (normally you wouldn't see this in silo.log, but would observe it on stdout when trying to run a process from the command line to debug it), you can raise the memory limit as a work around.

To set the memory limit of em7 process X to Y bytes, add a section like this to /etc/silo.conf (config push is an exception to this, see [memory usage, setting a limit on config push]):

```
[PROC_VIRTUAL_MEM_LIMIT]
X=Y

```

You can find the EM7 process IDs for a given process by going through the GUI to system → settings → processes and looking at the ID column, or, by going to this file in a core EM7 GIT working tree:

```
G3code/sql/datasets/init_data/init_state/master/system_settings_procs.sql

```

and examining individual lines like (which tells us that the process id for "Enterprise Database: Dynamic App Stats Processing" is 95):

```
INSERT INTO `system_settings_procs` (`aid`, `name`, `factor`, `execution`, `date_create`, `date_edit`, `user_edit`, `frequency`, `throttle`, `state`, `offset`, `debug`, `control`, `systemd_control`, `process_throttle`, `mp_cpu_percent`) VALUES (95,'Enterprise Database: Dynamic App Stats Processing',0,'da_deviation_crunch.py','2010-06-24 14:28:35','2010-06-24 14:28:35',1,60,1,1,10,0,'','',30,75);

```

Sometimes a process that overruns its memory limit will raise an exception like this (reported in silo.log):

```
"ERROR","29849","83","proc_pda","252","MemoryError Exception, exiting"

```

and sometimes it will manifest as a segmentation fault from a manual run attempt on the command line, as in:

```
[root@usviaelomondb01 em7admin]# sudo -u s-em7-core SILO_DEBUG=1 /opt/em7/backend/da_deviation_crunch.py
Segmentation fault

```

segfaults: enabling core dumps

You can tell when segfaults are occurring on a system when you see messages like these in the kernel ring buffer (output of the dmesg command) or in the /var/log/kern file:

```
[ 4781.066114] python2.7[9966]: segfault at 8 ip 00007fb6ac4047b9 sp 00007fff27be0330 error 4 in libpython2.7.so.1.0[7fb6ac37d000+17e000]
[ 4781.338218] python2.7[11686]: segfault at 8 ip 00007fb6ac4047b9 sp 00007fb68da0d5a0 error 4 in libpython2.7.so.1.0[7fb6ac37d000+17e000]
[ 4781.530832] python2.7[11693]: segfault at 8 ip 00007fb6ac4047b9 sp 00007fb68da0d5a0 error 4 in libpython2.7.so.1.0[7fb6ac37d000+17e000]
[ 4781.759644] python2.7[11701]: segfault at 8 ip 00007fb6ac4047b9 sp 00007fb68da0d5a0 error 4 in libpython2.7.so.1.0[7fb6ac37d000+17e000]
```

The following procedures will allow core dumps to be created which you can then run gdb on to, among other things, get a backtrace after a segfault. By allowing proc_mgr.py to create core dumps, you will be allowing most em7 processes started by it to create core dumps as well.

for manually run processes

This is very straightforward

- become the root user
- run “ulimit -c unlimited” to allow for unlimited coredump sizes
- run the target process manually as a normal user (which will inherit the unlimited coredump size):

```
sudo bash
ulimit -c unlimited
sudo -u s-em7-core /opt/em7/backend/some_process.py
```

If you wanted to test this on a long running process, and the PID were X, you could go through the above steps in one login and then kill the process by sending it a segmentation fault via

```
kill -SIGSEGV X
```

If the kernel core pattern is set to “core”

```
cat /proc/sys/kernel/core_pattern
core
```

Then you should see a coredump files created in the directory you ran some_process.py in, with the process id as an extension:

```
ls
```

```
core.9398  core.9823
```

for processes started via systemd

Most em7 processes are started via systemd, and enabling coredumps from those requires modifying systemd config files and forcing systemd to reload its config.

Create this file (note that you will have to create the directory em7.service.d and the file coredump.conf under it):

```
/etc/systemd/system/em7.service.d/coredump.conf
```

in newer versions of em7, that file should be:

```
/usr/lib/systemd/system/em7.service.d/coredump.conf
```

with contents:

```
[Service]
LimitCORE=infinity

```

Alternatively, you could also add those lines to the file /lib/systemd/system/em7.service, but creating a file in /etc has the advantage of separating your temporary changes from the reference file that defines the em7.service attributes and should be considered a general best practice.

and then run:

```
systemctl daemon-reload
systemctl restart em7.service
```

Next, set the kernel's core_pattern to write coredumps to /tmp like this:

```
echo "/tmp/core_pid_%p_executable_%e_unixtime_%t" > /proc/sys/kernel/core_pattern
```

After you are done gathering core dumps, be sure to reset the core_pattern as below (or you will risk filling /tmp with corefiles before you know it) and remove the file you created in /etc:

```
# oracle linux sets this to "core" by default but this JIRA story (EM-56205) has modified that to "|/bin/false" as a defensive security measure.
echo "|/bin/false" > /proc/sys/kernel/core_pattern

# if this file already existed then just comment out your changes but otherwise
rm /etc/systemd/system/em7.service.d/coredump.conf
```

You should then be able to see coredumps appearing in /tmp like this:

```
[root@cu1 em7admin]# ls -la /tmp/core*
-rw------- 1 s-em7-core s-em7-core 41529344 Jun  3 18:23 /tmp/core_pid_16126_executable_python2.7_unixtime_1591208588
-rw------- 1 s-em7-core s-em7-core 41529344 Jun  3 18:24 /tmp/core_pid_16405_executable_python2.7_unixtime_1591208647
```

```
-rw------- 1 s-em7-core s-em7-core 41529344 Jun  3 18:25 /tmp/core_pid_16673_executable_python2.7_unixtime_1591208707

```

Note that each of these are 44MB. They will fill up all available space in /tmp before you know it. Be sure to

- reset the core_pattern
- revert the /etc changes
- delete the core files from /tmp after you retrieve them

When analyzing a core dump, transfer it to a test stack with the same version of em7 as the customer and then:

```
# gdb is not installed by default on appliances
yum install gdb

# the general form is: gdb <executable file> <core dump produced by a process running that executable>
gdb $(which python2.7) core_pid_16126_executable_python2.7_unixtime_1591208588
```

analyzing core dump files

Here is a tutorial on how to do this in EM7: [Fasthosts Internet Alerting Outage#2021-08-06ATutorialonAnalyzingCoredumpsfromSystemLibrariesUsingGDB]

Oracle does not publish the debuginfos package we need to debug python

Update: I believe I have worked out a procedure to reliably compile the debuginfos packages for python here (see the next section).

Unfortunately, Oracle does not provide the debuginfos rpm package we would need to debug python code using coredumps gathered from customer systems. Recent versions of oracle linux 7 use a version of python for which Oracle does not publish the corresponding debuginfos package.

I have written a Jira issue to address this: EM-54559: lack of ol7 python2.7 debuginfos packages limits our ability to investigate coredumps CLOSED

Here are some escalations that where segfaults were involved

- EM-56998: Need to be address Significant segfaults in the /var/log/kern file on TCS DDP EU CLOSED (php segfault, but this obviously relates)
- EM-56406: Dynamic App Stats Processing Segfaults CLOSED
- EM-53873: Segfaults caused by dcmr_rebuild.py CLOSED

And here are escalations where a process was hanging and, if we had the debuginfos, we could enable coredumps, send a segv signal to the process, and then be able to determine the exact python line of code where the process was hanging:

- EM7 Process Manager | 2022 11 17 EM 54877 Proc_mgr hung in DE / RDS system on multiple clients

a procedure to compile python debug packages

See this article: oracle linux: compiling python debuginfo packages from python source packages | 2023 08 10 building python for oracle linux 7 using mock, then debugging core f...

segfaults: faulthandler.enable()

Here is a case where the techniques described here were used: [Fasthosts Internet Alerting Outage#2020-05-29Analysisofsegfault.log%3Anetsnmplibraryissegfaulting]

I believe you would have a hard time analyzing a core file with gdb (I mean unless we have access to the oracle debuginfos package for python) from a python program that segfaulted and figuring out from that, which line of python code was in-progress when some library the interpreter made a call into segfaulted. The faulthandler library offers a simple means of figuring that out.

Here is a test program showing how this works:

```
#!/usr/bin/env python
# file_summary: calling faulthandler.enable() will allow you to get a backtrace showing which line of code caused the segfault

import faulthandler
import ctypes

def g():
   ctypes.string_at(0)

def f():
   g()

faulthandler.enable()
g()

```

running this produces:

```
./test1.py
Fatal Python error: Segmentation fault

Current thread 0x00007f6448d87740 <python> (most recent call first):
  File "/usr/lib/python2.7/ctypes/__init__.py", line 510 in string_at
  File "./test1.py", line 8 in g
  File "./test1.py", line 14 in <module>
```

```
Segmentation fault (core dumped)
```

In practice (when applying to complex programs), you would add a line like this:

```
faulthandler.enable(file=open("somefile","a"))

```

and expect so see the tracebacks written there.

You might think this would work to log tracebacks from dynamic collect (editing the file /opt/em7/backend/dynamic_collect.py on a customer system):

```
#!/opt/em7/bin/python
# SILO_COPYRIGHT

################################################################################
#
# Module: dynamic_collect.py
# Author: Matthew Luebke
#
################################################################################

import os

from silo_common.database import silo_cursor
from silo_common.logger import silo_logger
from silo_envs.env_manager import EnvManager
from silo_collect.collect_dynamic import collect_dynamic, collect_dynamic_env

# add these lines
import faulthandler
faulthandler.enable(open("/tmp/segfault.log","a"))

dbc = silo_cursor.local_db()
if EnvManager.collection_environments_enabled(dbc):
    collect_dynamic()
else:
    debug = os.environ.get('SILO_DEBUG', 0)
    logger = silo_logger.get_logger_instance(dbc=dbc, arg_debug=debug, arg_interactive=True)
    eman = EnvManager(dbc, logger)
    eman.deploy_envs()
    collect_dynamic_env()

```

but I tested this and it doesn't work. By moving the faulthandler.enable() call around, I determined that placing the call inside of G3code/backend/process/proc_pds.py:process_pda.__init__() this works (I was working on tag EM7_G3_8.14.4 when I wrote this):

```
diff --git a/G3code/backend/process/proc_pda.py b/G3code/backend/process/proc_pda.py
index ae7b2bc4f8..baee1761e6 100755
--- a/G3code/backend/process/proc_pda.py
+++ b/G3code/backend/process/proc_pda.py
@@ -83,6 +83,14 @@ def proc_limit(frequency, factor):
     else:
```

```
        return int(factor)

+def enable_segfault_tracebacks(config):
+    """
+        uses faulthandler library to enable logging of python tracebacks when a segfault is received
+    """
+    logfile= config.get("LOCAL", "segfault_logfile")
+    if logfile:
+        import faulthandler
+        faulthandler.enable(file= open(logfile, "a"))

 class process_pda(object):
     """
@@ -139,6 +147,7 @@ class process_pda(object):

             self.model_type = int(self.config.get("LOCAL", "model_type"))
             self.model_name = self.config.get("LOCAL", "model")
+            enable_segfault_tracebacks(self.config)

             self.dbc = self._get_own_dbc()
             assert(isinstance(self.dbc, silo_cursor))

```

If you were to prepare a proc_pda.pyc for a given version and drop it on the customer system at:

```
/opt/em7/backend/process/proc_pds.pyc

```

and then add this to /etc/silo.conf:

```
[LOCAL]
segfault_logfile = /tmp/segfault.log
```

you could expect any segfaulting processes to have their tracebacks written to that log file.

The call is placed there because all scheduled EM7 processes inherit from that class, so this would enable a segfault in almost any EM7 process to be logged.

There is no need to restart anything to turn logging on or off. You can simply uncomment / comment out the logfile definition in /etc/silo.conf

running in debug mode and gathering logs

Procedure (thanks to @Jason Clark for writing this out in EM-62397):

```
Frist mask the systemd unit so it doesn’t try to auto start
   systemctl mask em7

Then run in debug
   sudo -u s-em7-core bash -c "SILO_DEBUG=1 /opt/em7/backend/proc_mgr.py"

It may produce an obvious error but if not, after it crashes collect any output to the screen any any logs in /var/log/sl1 and /var/log/em7 for review.
After collecting the logs unmask the unit
```

```
  systemctl unmask em7
```

Process Manager Hangs after drbd HA switchover

This started happening after the python2 to python3 change. Here is a case where this happened: [EM7 Process Manager - Engineering - Confluence]

The problem is that, after the switch over

- the old passive / new active CDB's process manager is trying to communicate with the old active / new passive mariadb
- without any reasonable timeout being set and therefore
- the goverening TCP/IP timeouts are in effect which
- takes a long time

As of 2025-07-09, systems engineering had a fix for this in QA

Theory

systemctl details

The service file for this process lives at /lib/systemd/system/em7.service

As of version 8.13.0, the service file is:

```
#
# EM7 Init
#

[Unit]
Description=EM7 Process Manager
After=network.target
After=syslog.target
After=mysqld.service

# snmptrapd wants the DB and schema to be available
Before=snmptrapd.service
# EM7 needs to be licensed and started before powerpack can be installed
Before=em7-powerpack-install.service

[Install]
WantedBy=multi-user.target

[Service]
Type=simple
User=s-em7-core
Group=s-em7-core
ExecStartPre=/opt/em7/share/scripts/em7_firstboot.sh
ExecStartPre=/opt/em7/share/scripts/em7_init.sh
ExecStart=/opt/em7/backend/proc_mgr.py
TimeoutStartSec=900
Restart=always
```

```
RestartSec=1
PermissionsStartOnly=true
SuccessExitStatus=143
StandardOutput=null
StandardError=null

# systemd should never give up trying to restart proc_mgr
StartLimitInterval=0
```

process memory limits

In G3code/backend/process/proc_pda.py we have:

```
class process_pda(object):
...
    def __init__(self, sp_id, daemonize=0, do_pid=False, set_limit=(1 << 30), group_id=0, debug=0):
...
            mem_limit = self.mem_limit_config()
            if mem_limit:
               set_limit = mem_limit
            if set_limit:
                # Limit each Python process to a suitable address space
                # This is important to prevent a runaway process bringing
                # the box to its knees - default is 350M if not set
                try:
                    resource.setrlimit(resource.RLIMIT_AS,(set_limit, resource.getrlimit(resource.RLIMIT_AS)[1]))
                except ValueError:
                    pass
...

```

So, by default, a process will have 1024 MB or 1GB (despite the comment's claiming the default is 350MB) for its address space limit (which for a single threaded process, functions as its memory limit).

```
python
Python 2.7.17 (default, Apr 15 2020, 17:20:14)
[GCC 7.5.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> (1<<30)/(1024*1024.0)
1024.0

```

And we can see in mem_limit_config():

```
    def mem_limit_config(self):
        # get process virtual memory limit from silo.conf to overwrite default value .
        try:
            proc_mem_limit = self.config.get("PROC_VIRTUAL_MEM_LIMIT", self.sp_id)
            self.logger.debug("Got proc virtual memory limit from silo.conf:%s" % (proc_mem_limit))
            if proc_mem_limit and isinstance(proc_mem_limit, str) and re.match(r'^\d*$', proc_mem_limit):
                proc_mem_limit = int(proc_mem_limit)
```

```
                if int(proc_mem_limit) and proc_mem_limit >= 1073741824 :
                    self.logger.debug("Set Process virtual memory limit from silo.conf:proc_id=%s : %s"
                                      % (self.sp_id, proc_mem_limit))
                    return proc_mem_limit
                else:
                    self.logger.debug("Memory limit overwrite rejected,It should be more than 1GB:proc_id=%s : %s"
                                      % (self.sp_id, proc_mem_limit))
            else:
                self.logger.debug("No or Incorrect virtual memory limit from silo.conf:proc_id=%s : %s"
                                  % (self.sp_id, proc_mem_limit))
        except (ValueError, TypeError) as e:
            self.logger.debug("Error, Process virtual memory limit from silo.conf:proc_id=%s: %s"
                              % (self.sp_id, e))
        except Exception as e:
            self.logger.debug("Memory limit overwrite failed due to exception:proc_id= %s: %s"
                              % (self.sp_id, e))
        return 0

```

that you can override the default, for a given process's em7 process_id X, to be Y bytes by editing /etc/silo.conf to contain:

```
[PROC_VIRTUAL_MEM_LIMIT]
X=Y
```

By doing a find+grep search through the backend code working tree we can see that config_push, data_pull, and c_task_manager override this mechanism to set different limits:

```
./silo_db/config_push/config_push.py:232:            resource.setrlimit(resource.RLIMIT_AS,
./silo_db/data_pull/main.py:631:            resource.setrlimit(resource.RLIMIT_AS, (config["memory_limit"], resource.getrlimit(resource.RLIMIT_AS)[1]))
./silo_db/c_task_manager.py:621:            resource.setrlimit(resource.RLIMIT_AS, (v, resource.getrlimit(resource.RLIMIT_AS)[1]))
./silo_common/report_run.py:99:            resource.setrlimit(resource.RLIMIT_AS,(memlimit, resource.getrlimit(resource.RLIMIT_AS)[1]))
./old_unittest_data/test/test_profile.py:13:resource.setrlimit(resource.RLIMIT_AS, (700000000,700000000))
./process/proc_driver.py:460:                resource.setrlimit(resource.RLIMIT_AS, (oldrlimit[1], oldrlimit[1]))
./process/proc_driver.py:481:                resource.setrlimit(resource.RLIMIT_AS, oldrlimit)
./process/proc_pda.py:177:                    resource.setrlimit(resource.RLIMIT_AS, (set_limit, resource.getrlimit(resource.RLIMIT_AS)[1]))

```

Time Factor

Picture a process that runs every 5 minutes. You would expect that, in a healthy system, it would take no longer than 5 minutes to run. However, suppose you wanted to allow a process to

run up to 3 times as long as it normally should, in this case for 15 minutes. You could implement that for a process by setting the time factor to 2. So the time factor is the number of additional normal-runtimes you want it allowed to run before its killed for taking too long. In other words, allowed runtime in minutes = (frequency in minutes) * (time_factor + 1).

Stderr for scheduled processes (/data/logs/X.py.log files and how they grow without limit)

When the em7 process manager starts a scheduled process (the classic example being collection processes), the code in this file:class.method(), G3code/backend/process/proc_driver.py.p_driver(process.process_pda).proc_scheduled() is called:

```
...
LOGFILE_FMT = "/usr/local/silo/data/logs/%s.log"
...
class p_driver(process.process_pda):
...
    def proc_scheduled(self):
...
            process_log_file = LOGFILE_FMT % (record['execution'],)
            try:
                process_logger = open(process_log_file, mode='a+')
            except IOError, (errno, strerror):
                self.logger.error("Could not open process log file %s for writing: I/O error (%s): %s", process_log_file, errno, strerror)
                process_logger = None
...
            try:
                # A process should always be started by this call,
                # though that process may end very quickly (e.g. if there is already one running and only 1 is allowed, etc).
                pid = subprocess.Popen(
                    [execution, "0", "0", "--datetime", str(dt)],
                    stderr=process_logger,
                    preexec_fn=os.setpgrp,
                    env=dict(os.environ, SILO_DEBUG=os.environ.get('SILO_DEBUG', str(debug)))
                ).pid
...
```

So for a process like maint_hourly.py, we would expect stderr messages to be written to:

```
/usr/local/silo/data/logs/maint_hourly.py.log
```

The usual mechanism by which lines are written to an em7 python process's stderr is (in file G3code/backend/process/proc_pda.py):

```
class process_pda(object):
...
    def __init__(self, sp_id, daemonize=0, do_pid=False, set_limit=(1 << 30), group_id=0, debug=0):
...
        try:
```

```
...
            self.main()
...
        except Exception, e:
            if self.logger != None:
                self.log_exception(e, self.dbc, self.logger) # Record debug info (DB state, etc) before we log our error msg.
                self.logger.critical(
                    "Unhandled exception on appliance: %s, process %s:",
                    self.mid,
                    self.proc_name,
                    exc_info=True
                )

            self.close(
                exit_status=1,
                exit_msg="Unhandled exception: {}".format(traceback.format_exc()),
                log_exit_msg=False  # we've already raised a CRITICAL-level stink
            )
...
   def close(self, exit_status=0, exit_msg="", log_exit_msg=True):
...
        try:
            if exit_status != 0:
                if not exit_msg:
                    exit_msg = "Unknown error in %s caused exit status of %s"
                    exit_msg %= (self.proc_name, exit_status)
                print(exit_msg, file=sys.stderr)
                if log_exit_msg:
                    self.logger.error(exit_msg)
            self.logger.debug("Logging stopped for %s"% (self.proc_name))
        except Exception:
            pass
...
```

Walking through the above code, after an unhandled exception occurs

- we arrive at the except clause in “__init__()” and call self.close() where
- exit_msg is written to sys.stderr

Because this process had its stderr redirected to /usr/local/silo/data/logs/X.py.log, and that file was opened in mode append:

```
                process_logger = open(process_log_file, mode='a+')
```

these files will grow without limit. Possible fixes to this include

- changing the open mode to “w” (which would limit the usefulness of capturing stderr)
- introducing a maintenance process that trims any /data/logs/X.py.log files to a maximum size
- replacing the open() line of code above with the creation of an object that rotates log files

I have attached a git bundle of the research branch where marked up the code quoted above:

research_bundle
20 Dec 2023, 08:54 PM

You can pull the research branch from the bundle file like this:

```
git remote add stderr /tmp/research_bundle
git ls-remote stderr
e34047ffadfaee6d4dbdbd2c283d4c31b67457df refs/heads/research_process_stderr_capture_in_data_logs
git fetch stderr research_process_stderr_capture_in_data_logs:research_process_stderr_capture_in_data_logs
git checkout research_process_stderr_capture_in_data_logs
```

The FIXME_INFO commentary follows this scheme: A Proposed Project Documentation Practice: FIXME Commentary

The fixme report reads:

```
./backend/process/proc_pda.py:95:# FIXME_INFO_CLASS: process_pda(object) {
./backend/process/proc_pda.py:120:# FIXME_INFO_FN: __init__() {
./backend/process/proc_pda.py:269:# FIXME_INFO_ISSUE_1_9: from various exception catching clauses here.  This clause handles process UHEs and passes the traceback in through the exit_msg
./backend/process/proc_pda.py:275:# FIXME_INFO_FN: } __init__()
./backend/process/proc_pda.py:306:# FIXME_INFO_FN: close() {
./backend/process/proc_pda.py:341:# FIXME_INFO_ISSUE_1_1: the exit_msg for a process, like config_push, is written to /data/logs/config_push.py.log.  How does that happen?
./backend/process/proc_pda.py:342:# FIXME_INFO_ISSUE_1_2: we see the message being written to stderr (and to logger.error) here.
./backend/process/proc_pda.py:343:# FIXME_INFO_ISSUE_1_8: returning to where the question started, the exit_msg, written to stderr here, is passed in
./backend/process/proc_pda.py:362:# FIXME_INFO_FN: } close()
./backend/process/proc_pda.py:617:# FIXME_INFO_CLASS: }process_pda(object)
./backend/process/proc_driver.py:64:# FIXME_INFO_ISSUE_1_6: derived from this string template LOGFILE_FMT is set to "/usr/local/silo/data/logs/%s.log" here and record['execution'] comes from
./backend/process/proc_driver.py:174:# FIXME_INFO_CLASS: p_driver(process.process_pda) {
./backend/process/proc_driver.py:638:# FIXME_INFO_FN: proc_scheduled() {
./backend/process/proc_driver.py:671:# FIXME_INFO_ISSUE_1_5: using this filename:
./backend/process/proc_driver.py:674:# FIXME_INFO_ISSUE_1_4: which is just a file opened here
./backend/process/proc_driver.py:675:# FIXME_INFO_ISSUE_1_10: becuase of how the log file is opened here, these files can grow without limit.  how can this be fixed?
./backend/process/proc_driver.py:676:# FIXME_INFO_ISSUE_1_11: proposal 1: instantiate a log rotation object here, that sets a size limit
./backend/process/proc_driver.py:677:# FIXME_INFO_ISSUE_1_12: proposal 2: write an hourly maintenance task that only leaves the most recent X MB worth of text lines
./backend/process/proc_driver.py:678:# FIXME_INFO_ISSUE_1_13: proposal 3: rewrite with mode="w"
./backend/process/proc_driver.py:689:# FIXME_INFO_ISSUE_1_3: here was the command line that started the process, and stderr was set to "process_logger"
./backend/process/proc_driver.py:702:# FIXME_INFO_FN: } proc_scheduled()
./backend/process/proc_driver.py:1030:# FIXME_INFO_CLASS: } p_driver(process.process_pda)
./sql/datasets/init_data/init_state/master/system_settings_procs.sql:15:/* FIXME_INFO_FILE */
./sql/datasets/init_data/init_state/master/system_settings_procs.sql:16:/* FIXME_INFO_ISSUE_1_7: from the records in master.system_settings_procs here, as in
```

```
maint_freq.py here*/

```

Post Mortem Articles

2025-06-24 EM-74711 EM7 service active but not functional after HA failover

Link:[EM-74711] EM7 service active but not functional after HA failover - Jira

From the description:

```
Observed Behavior: proc_mgr fails during failover and requires customer to start it manually.

Expected Behavior: proc_mgr should not fail during failover.

Summary of troubleshooting steps taken so far and the current status:
Customer is currently experiencing DB hung issue due to unknown reasons(EM-74637: System hangs with rcu_sched CPU stall, hung task, and soft lockup after updating to 12.3.1 from 12.1.2
Tier 3
 ).
At times when Primary DB hangs, failover completes to the Secondary node, but proc_mgr is not managing/starting processes on it.
When checking /var/log/em7/em7.log files, found these errors:

Error 1:
Current thread 0x00007fb4c2544b80 (most recent call first):
  File "/opt/em7/lib/python3.6/proc_driver.py", line 443 in main
  File "/opt/em7/lib/python3.6/proc_pda.py", line 222 in __init__
  File "/opt/em7/lib/python3.6/proc_driver.py", line 1311 in proc_driver
  File "/opt/em7/backend/proc_mgr.py", line 38 in <module>

Another error:
Current thread 0x00007ff8f591ab80 (most recent call first):
  File "/opt/em7/lib/python3/MySQLdb/connections.py", line 254 in query
  File "/opt/em7/lib/python3/MySQLdb/cursors.py", line 319 in _query
  File "/opt/em7/lib/python3/MySQLdb/cursors.py", line 206 in execute
  File "/opt/em7/lib/python3.6/cursor.py", line 464 in execute
  File "/opt/em7/lib/python3.6/cursor.py", line 99 in callFunc
  File "/opt/em7/lib/python3.6/cursor.py", line 120 in wrapper
  File "/opt/em7/lib/python3.6/cursor.py", line 766 in execute_d
  File "/opt/em7/lib/python3.6/cursor.py", line 87 in callFunc
  File "/opt/em7/lib/python3.6/cursor.py", line 99 in callFunc
  File "/opt/em7/lib/python3.6/proc_driver.py", line 185 in get_spool_records_v2
  File "/opt/em7/lib/python3.6/proc_driver.py", line 500 in proc_async
  File "/opt/em7/lib/python3.6/proc_driver.py", line 260 in do_every
  File "/opt/em7/lib/python3.6/proc_driver.py", line 457 in main
  File "/opt/em7/lib/python3.6/proc_pda.py", line 222 in __init__
  File "/opt/em7/lib/python3.6/proc_driver.py", line 1311 in proc_driver
  File "/opt/em7/backend/proc_mgr.py", line 38 in <module>
When checking the status, it appears to be running, but not doing anything. However when customer manually restarts it, it starts managing processes correctly. Here is the status that shows as if the process is running, but at the same time system_status.sh shows that it is hung:

```

```
[root@edtnab02em54 em7admin]# systemctl status em7
â—• em7.service - EM7 Process Manager
   Loaded: loaded (/usr/lib/systemd/system/em7.service; enabled; vendor preset: disabled)
   Active: active (running) since Sat 2025-06-14 20:16:13 UTC; 3h 57min ago
  Process: 2076 ExecStartPre=/opt/em7/share/scripts/em7_init.sh (code=exited, status=0/SUCCESS)
  Process: 2064 ExecStartPre=/opt/em7/share/scripts/em7_firstboot.sh (code=exited, status=0/SUCCESS)
 Main PID: 2492 (proc_mgr.py)
    Tasks: 1 (limit: 3355442)
   Memory: 176.3M
   CGroup: /system.slice/em7.service
           â””â”€2492 /opt/em7/bin/python3 /opt/em7/backend/proc_mgr.py

Jun 14 20:16:12 edtnab02em54 systemd[1]: Starting EM7 Process Manager...
Jun 14 20:16:13 edtnab02em54 systemd[1]: Started EM7 Process Manager.
[root@edtnab02em54 em7admin]# system_status.sh


After the service is restarted suing "systemctl restart em7" all processes start properly.
Could someone please help find out the root cause of the issue and if there is a way to prevent it from happening?
Debug logs attached.
Thank you.
-Leo
```

Mark Merriken wrote this comment: EM-74711: EM7 service active but not functional after HA failover SUPPORT

which we quote below:

```
Thanks @Chris Marshall for these steps.

Given a few other cases with similar symptoms, we’ve gotten evidence of, and have recreated, a situation whereby the remaining HA DB (the passive one) becomes primary, but proc_mgr gets “stuck” in its main loop while executing DB queries to a no-longer-alive database server. The cursor doesn’t time out and remains waiting until the OS/kernel-level TCP socket timeouts kick in and eventually cause an exception within proc_mgr which is then immediately restarted by systemd.

We have a tentative fix for this in QA now.

As for a cause (and a “why is this now a thing when not really before?”) we suspect there might have been some timing parameters “missed” when doing the python2 to python3 migration of the core database cursor libraries used by proc_mgr and other python processes.

Thanks

#salesforce
```

2024-08-20 EM-66942: em7 service gets hung after HA failover, doesn't restart for 15 minutes delaying other necessary services from starting

Link:[EM-66492] em7 service gets hung after HA failover, doesn't restart for 15 minutes delaying other necessary services from starting - Jira

From the description:

```
Observed Behavior: After hard stopping the HA primary, the em7 service appears to start on the other HA node, but then gets hung. About 15 minutes later it restarts and is able to start all the other services, like em7_event

Expected Behavior: It doesn't get hung the first time it starts, the other services start sooner than 15+ minutes later, so the product remains functional in a high availability failover

Summary of troubleshooting steps taken so far and the current status:
Tried to put the em7 process in debug to determine why it is getting hung, but the debug logs are not really showing any information.
Asked in a bunch of eng channels:
https://sciencelogic.slack.com/archives/C8XHFKRB7/p1724178435401989
https://sciencelogic.slack.com/archives/C1C2GTCAZ/p1724180664996849
https://sciencelogic.slack.com/archives/C04L444TLG2/p1724183550231199

This seems to happen in 11.3 too and STIG/MUD settings don’t appear to be the issue. This probably crops up because not everyone uses a
crossover cable between HA nodes so there isn’t a link down event on the new primary to bring the interface down and thus close any stale
connections to the old primary’s keep alive IP. This causes nearly 15-17 minutes of lag time until proc_mgr is restarted by systemd, at
which time it successfully connects to the LOCAL dbaddr and takes over successfully.

We should investigate potential ways of getting the em7.service bounced on the secondary after it takes over.
```

2022-11-17 EM-54877 Proc_mgr hung in DE / RDS system on multiple clients

Link: proc_mgr.py hung in DE / RDS configuration

In this case, proc_mgr.py is getting hung, intermittently, on several customers. This happens every few days at some customer sites. In addition to hanging, we see long delays between the mysql requests of 300 seconds or more in the logs analyzed for Nokia. As proc_mgr is supposed to check for which processes to run every minute, a pause of five minutes between queries means some non-query operation proc_mgr is trying to do is taking way too long.

2022-11-11 NIH segfault topoplogy crunch

Link: NIH-CIT dcmr_rebuild.py segfault

In this case, we originally thought the dcmr_rebuild process was receiving the segfaults, but upon further analysis we determined it was the topology crunch process

2020-05-02 Symmetry: RBA proc fails II

Link here: symmerty rba proc fails II

Similar to the first time. A new ingrediant for me was the role of the pid file in preventing patch manager for starting up again.

2020-01-17 Symmetry: RBA proc fails

Link here: [Symmetry: RBA proc fails]

In this outage, one of the per minute processes that proc_mgr.py was starting via a systemctl start command, em7_patch_manager, was hung which in turn caused the systemctl start command that proc_mgr was using to start it to hang, which in turn hung proc_mgr.py itself.

2020-02-19 eLoyalty: std dev alerting and DCM deadlocks

Link here: [std dev alerting and DCM deadlocks]

In this outage, the work around was to raise the memory limit on da_deviation_crunch.py to 2GB, as detailed in the section on memory errors above. As I write this, the first attempt failed.

Known Issues

EM-34767: blocking subshell call to systemctl start

This is detailed here: [2020-01-20 proc_mgr.py has a defect: should not block waiting for systemctl start to return]

The Jira ticket EM-34767 has been filed to correct this.